



BrilliantDex

Decode Your Chess. Discover Your Brilliance.

PROJECT DOCUMENTATION & PERFORMANCE INSIGHTS

4 – 0

RECORD

100%

WIN RATE

89.9

BRILLIANTDEX SCORE

997

AVG. OPPONENT ELO

Player: brilliancext · Chess.com · 4 Games Analyzed · 10-Minute Rapid
Built with Python · Pandas · NumPy · Matplotlib · Seaborn · python-chess · Regex

About the Project

BrilliantDex is an end-to-end chess performance analytics platform built entirely in Python. It takes raw PGN (Portable Game Notation) game exports and turns them into structured, meaningful insight — surfacing a player's strengths, weaknesses, and playing patterns through automated analysis and visualization rather than manual review.

This run of the pipeline analyzes 4 real 10-minute rapid games played by **brilliantcext** on Chess.com, carrying each game from raw PGN text all the way through validation, parsing, feature extraction, visualization, and a final composite performance score.

Project Objectives

- Import one or multiple PGN games
- Clean and validate raw chess data
- Extract meaningful features from every game
- Perform exploratory data analysis (EDA)
- Visualize chess performance through charts
- Compare wins, draws, and losses
- Analyze White vs. Black performance
- Generate improvement insights
- Calculate an overall BrilliantDex Performance Score

Player Snapshot

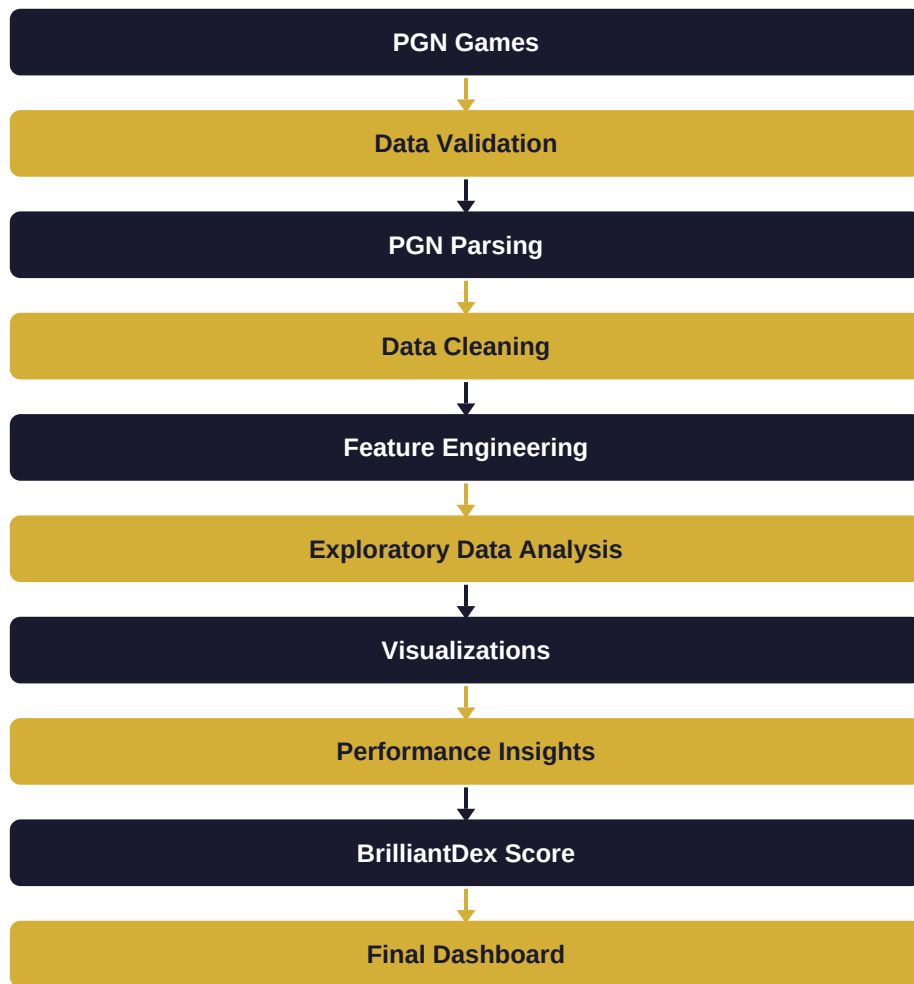
Username	brilliantcext
Platform	Chess.com
Time Control	600s · 10-Minute Rapid
Games Analyzed	4
Record	4W – 0D – 0L (100% win rate)
BrilliantDex Score	89.9 / 100

Tech Stack & Project Workflow

Technologies Used

Technology	Role In This Project
Python	Core language tying the entire pipeline together, start to finish.
Pandas	Structures every parsed game into a clean, analyzable feature table.
NumPy	Powers the numeric calculations behind the BrilliantDex Score.
Matplotlib	Renders the chart suite and the final 2×3 performance dashboard.
Seaborn	Adds the statistical visualizations — captures vs. checks, Elo-diff vs. game length.
python-chess	Replays every move of every game to detect checkmates, captures, checks, and castling.
Regex (re)	Validates raw PGN structure and fixes Chess.com's single-line header export quirk.

Pipeline Workflow



Methodology Deep-Dive

1. Data Validation

Before anything gets parsed, every raw PGN string is checked against a small set of required structural rules using regular expressions: it must contain [White, [Black, and [Result tags, plus a valid result token (1-0, 0-1, 1/2-1/2, or *). Games that fail validation are flagged rather than silently skipped.

2. PGN Parsing — and a Real Bug

This step surfaced a genuine real-world data quality issue. Chess.com's PGN export crams every header tag onto a single line instead of one tag per line, the format python-chess's header parser expects. Fed directly, the parser silently mis-reads the file — the White player becomes '?' and WhiteElo comes back as None, with no error raised. The fix is a small `normalize_pgn_headers()` function that inserts a line break before every header tag using regex, run after structural validation but before parsing. It's the kind of bug that's invisible until the output is checked — which is exactly why every parsed game is spot-checked against the source PGN.

3. Data Cleaning

Once parsed, headers are normalized: Elo ratings are cast to integers, missing values are handled explicitly rather than silently dropped, and player names are stripped of stray whitespace.

4. Feature Engineering

Every game is replayed move-by-move on a python-chess board. From that replay, 20 features are extracted per game, grouped below.

Identity & Setup: `game_id`, `white_player`, `black_player`, `opponent`, `user_color`, `time_control`

Rating & Outcome: `user_elo`, `opponent_elo`, `elo_diff`, `user_result`

Move-Level Stats: `total_full_moves`, `total_plies`, `total_captures`, `total_checks`, `total_promotions`

Special Moves: `ended_in_checkmate`, `user_castled`, `castle_side`, `opening_moves`, `first_move`

5. Exploratory Data Analysis

With a clean feature table in hand, the EDA step reviews summary statistics across the batch — average Elo faced, move-count distribution, capture and check intensity — before any of it becomes a chart.



The BrilliantDex Score

The BrilliantDex Score condenses everything the pipeline measured into a single 0–100 number, built from four weighted components:

Component	Weight	What It Measures
Win Rate Score	35%	Straightforward win percentage across the batch.
Clutch Factor	30%	Performance vs. Elo expectation, using the standard expected-score formula $1 / (1 + 10^{((\text{opponent_elo} - \text{user_elo})/400)})$ — rewards wins where the opponent was statistically favored.
Finishing Power	20%	Of the games won, the share that ended in an actual delivered checkmate rather than a resignation.
Aggression Index	15%	$(\text{Total Checks} + \text{Total Captures}) / \text{Total Moves}$, averaged across games — a proxy for how relentlessly the attack is pressed.

This Run's Breakdown

Component	Score
Win Rate Score	100.0 / 100
Clutch Factor	91.8 / 100
Finishing Power	75.0 / 100
Aggression Index	82.7 / 100
BRILLIANTDEX SCORE	89.9 / 100

Performance Tiers

Range	Tier	
90 – 100	 Grandmaster Brilliance	
75 – 89.9	 Tactical Powerhouse	← This Run
60 – 74.9	 Solid Contender	
40 – 59.9	 Developing Player	
0 – 39.9	 Just Getting Started	

At 89.9, this run lands solidly in **Tactical Powerhouse** territory — just 0.1 points shy of Grandmaster Brilliance. Win Rate and Clutch Factor are doing the heavy lifting; Finishing Power (75.0) is the one component with real room to grow, since one of the four wins ended by resignation rather than checkmate.

Insights & Findings

Game-by-Game Results

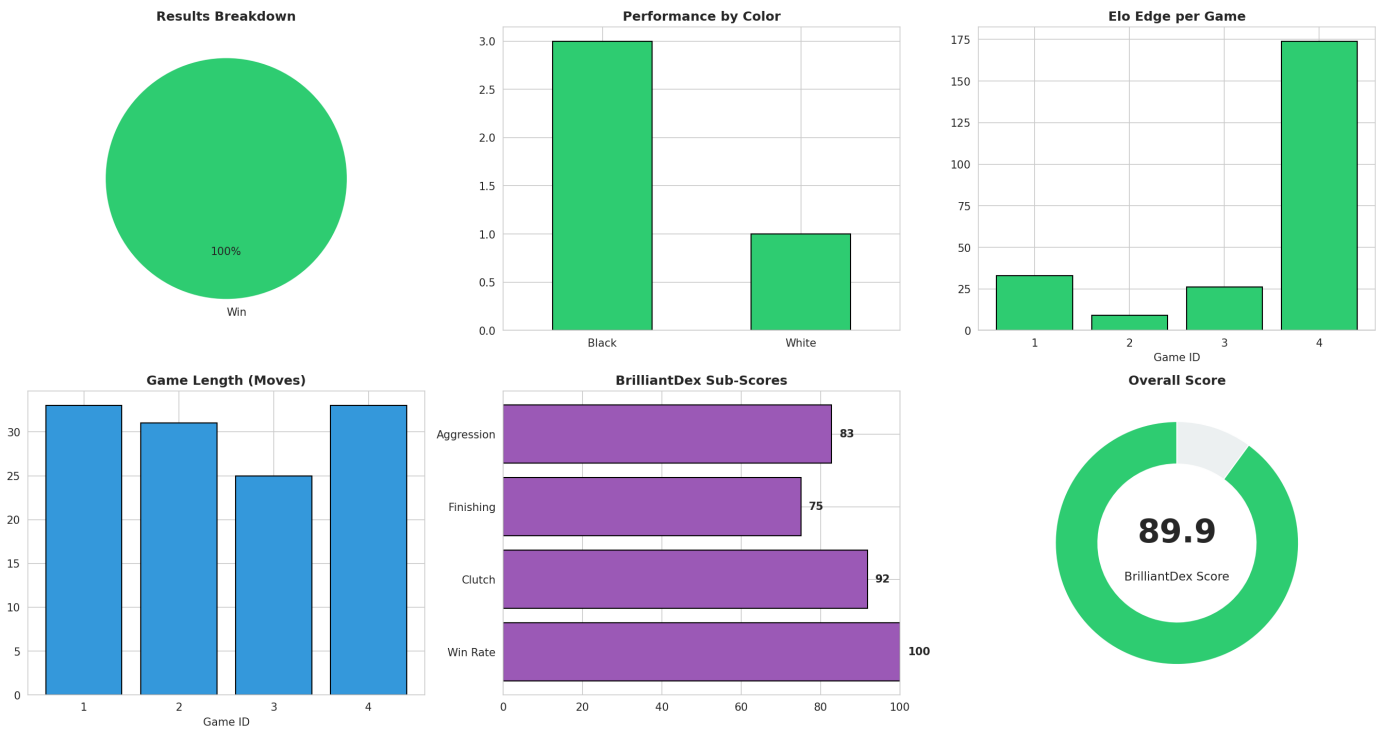
Game	Opponent	Color	Result	Elo Diff	Moves	Finish
1	Kkpdeep02	Black	Win	+33	33	Checkmate
2	Azezel6	Black	Win	+9	31	Checkmate
3	lovestock	Black	Win	+26	25	Resignation
4	Beetrootichigo	White	Win	+174	33	Checkmate

Key Findings

- Perfect 4–0 record across this batch — 100% win rate overall, and 100% in both color assignments (1/1 as White, 3/3 as Black).
- Average opponent Elo faced: **997**.
- Closest contest on paper: **Azezel6** (1091 Elo), just a +9 rating edge — the smallest gap in the batch, and the win came anyway.
- Most tactically active game: **Game 2 vs. Azezel6** — 19 captures and 13 checks, well above the batch average.
- Longest grind: Game 1 vs. Kkpdeep02 at 33 moves. Shortest: Game 3 vs. lovestock at 25 moves.
- 3 of 4 wins were finished with a delivered checkmate rather than a resignation — a strong finishing signal.
- Castled in 3 of 4 games — kingside every time it happened.

Final Performance Dashboard

BrilliantDex -- Final Performance Dashboard



How to Run & What's Next

Getting Started

1. Open **BrilliantDex_Chess_Analytics.ipynb** in Jupyter Notebook or JupyterLab.
2. Run all cells top to bottom (Cell → Run All). The first code cell installs and imports every dependency automatically.
3. To analyze a new batch of games, swap out the **raw_pgn**s list in the next code cell and update **USERNAME** if it's a different player — the rest of the pipeline re-runs unchanged.

Requirements

pandas · numpy · matplotlib · seaborn · python-chess · jupyter / ipykernel

What's Next

- Auto-fetch games directly from the Chess.com or Lichess public APIs instead of pasting PGNs by hand.
- ECO opening classification, so openings get named instead of shown as raw first-moves.
- Elo-over-time tracking once a larger batch of games is available to trend across.
- Per-piece activity stats — which pieces do the most work, square heatmaps.
- A lightweight Streamlit wrapper to turn this into a shareable, interactive dashboard.

 *BrilliantDex — Decode Your Chess. Discover Your Brilliance.* 